

AI-Powered Support Ticket Triage – Full-Stack Assignment (v4)

Objective

Build a small full-stack application that analyzes support tickets using simple AI-style logic, persists tickets in a database, and displays results in a UI. No external AI APIs are allowed.

Scope

- Backend REST API to analyze support tickets
- Database persistence for received tickets
- Simple Frontend UI to submit tickets and view results
- UI list view of previously analyzed tickets
- Local NLP / heuristic-based AI logic (no external APIs)
- Clean software engineering practices

Functional Requirements

- POST /tickets/analyze endpoint that accepts a ticket message
- Persist analyzed tickets to a database (SQLite/Postgres/MongoDB or equivalent)
- GET /tickets endpoint to list recent tickets
- Classify ticket into: Billing, Technical, Account, Feature Request, Other
- Assign priority: P0 (critical), P1 (high), P2 (medium), P3 (low)
- Detect urgency keywords and extract relevant keywords
- Return category, priority, signals, and confidence score

Frontend Requirements

- Text area to submit a support ticket
- Submit button to call backend API
- Result panel displaying category, priority, urgency, confidence, and keywords
- List view/table showing previously analyzed tickets (latest first)
- Loading and error states

AI / NLP Logic (Local Only)

- Keyword-based classification
- Urgency detection using predefined terms (e.g., urgent, asap, down)
- Priority scoring based on severity signals
- Confidence score based on keyword matches

Engineering Requirements

- Input validation and error handling

- Separation of concerns (controller, service, analyzer)
- Config-driven keyword rules
- Unit tests for classification and priority logic
- README with setup, architecture, and design notes
- Docker build for Backend and Frontend (docker-compose preferred)

Reflection Requirement (Required)

- Include a short reflection (max 1 page) in the README covering: design decisions, trade-offs, limitations, and what you would improve with more time.
- Explain why you chose your data model, API structure, and classification approach.

Customization Twist (Required)

- Add one custom classification or prioritization rule of your choice (e.g., special handling for 'refund' or 'security').
- Explain the rationale behind this custom rule in the README and demonstrate it in the demo video.

Submission Requirements

- Source code repository or zip (FE + BE)
- Working Docker build instructions (docker-compose up)
- Short demo video (2–5 minutes) showing end-to-end working app (including the custom rule)
- README with run instructions, design notes, reflection, and customization rationale
- Basic test results

Constraints

- No use of external AI/LLM APIs
- Must run locally via Docker
- Focus on correctness, reliability, and code quality over UI polish

Expected Effort

Approximately 5–7 hours.